DATA STRUCTURES & SYSTEMS DESIGN

An O(1) cache, and a scheduler that counts the peak.

Two problems, one habit: give each question to the structure that answers it cheaply, then prove the cost. Reasoning first, implementation second, measurements last.

CANDIDATE
Vihan Chorada

ROLE
AI / Multi-Agent Systems Intern

SUBMITTED
21 October 2026

STACK
Python 3.10 · standard library only

Contents overleaf · project layout and raw output in the appendix

Contents

Each problem follows the same shape: the reasoning in plain English, the implementation, the cases that break naive versions, and the evidence that the implementation handles them. Section 3 answers the four discussion questions directly.

1 **Problem 1 — O(1) LRU Cache** 3

1.1 Approach: two structures, one job each 3

1.2 The cache implementation 4

1.3 The cases that break naive versions 6

1.4 Verification of the cache 6

2 **Problem 2 — Event Scheduler** 7

2.1 One convention: half-open intervals 7

2.2 can_attend_all: is the diary free? 7

2.3 min_rooms_required: count the peak 8

2.4 The scheduler implementation 8

2.5 Verification of the scheduler 10

3 **Final Discussion & Analysis** 11

3.1 Time and space complexity 11

3.2 Trade-offs: why a hash map plus a doubly linked list 12

3.3 Future proofing: assigning room numbers 13

3.4 Concurrency: making the cache thread-safe 14

4 **Appendix** 16

4.1 Project layout and how to run 16

4.2 Captured output 16

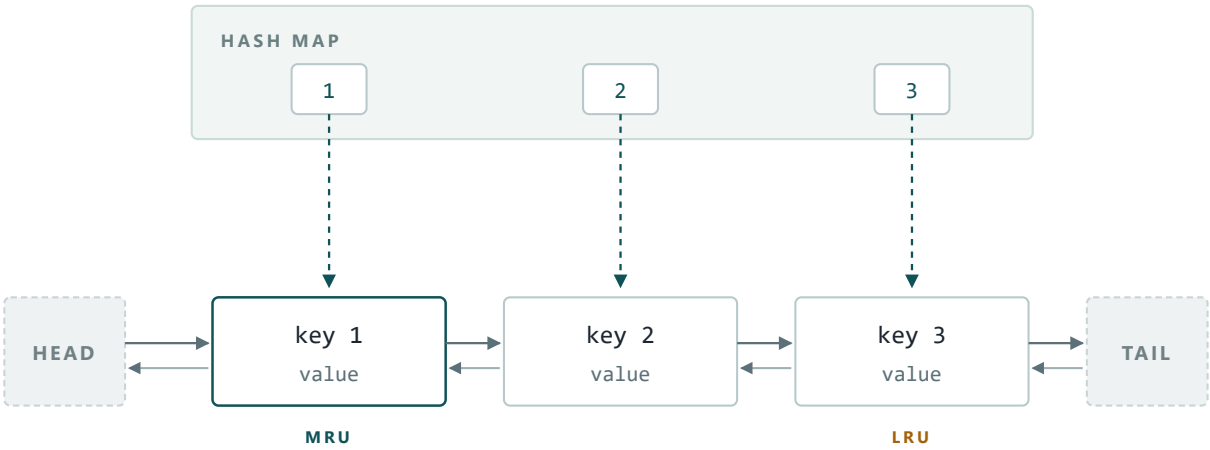
4.3 Test suite and benchmark 17

SECTION 1

Problem 1 — O(1) LRU Cache

Both operations must be constant time with respect to cache size, so neither may scan the cache. That single constraint decides the data structures.

> **THE SHORT VERSION**
>
> A hash map for lookup, a doubly linked list for order, and two sentinel nodes to delete the edge cases. Every operation touches a constant number of nodes no matter how full the cache is.



The two structures hold the same entries and are updated together. The map gives O(1) access to a node; the list gives O(1) reordering. Neither one alone can do both.

1.1 Approach: two structures, one job each

A cache is asked two different questions, and they are not the same question. The first is *given this key, where is its entry?* The second is *which entry has gone unused the longest?* A hash map is excellent at the first and has no opinion about the second. A linked list or an array is excellent at the second and needs a linear scan for the first. The whole exercise is about refusing to make one structure do both.

So I gave each question to the structure that answers it natively, and stored a link between them. The dictionary maps a key to the node that holds it, so a lookup never walks the list. The nodes form a doubly linked list in recency order, so once a node is in hand, moving it costs a fixed number of pointer writes. The dictionary is what makes `get` independent of how many entries the cache holds. The `prev` pointer is what makes removal independent of where the node sits.

Order runs from most recently used at the front to least recently used at the back. A hit in `get` moves its node to the front and returns the value; a miss returns -1 and changes nothing. `put` either overwrites an existing node and refreshes its position, or creates a node at the front — and if the dictionary was already at capacity, the node at `tail.prev` is unlinked and its key deleted from the dictionary. Each node stores its own key for exactly that last step: eviction holds a node, and turning a node back into a key would otherwise require a scan.

Both ends of the list are guarded by sentinel nodes that never hold data. They exist to remove branches. Without them, insertion and removal need separate cases for an empty list, for the head and for the tail — the three places where hand-written LRU caches usually break. With them, `head.next` and `tail.prev` always point

at a node object, and the same four assignments work at every position. Two extra objects buy a hot path with no conditionals.

A note on what I did not use. Python's `OrderedDict` is this exact structure implemented in C, and `functools.lru_cache` is the standard production answer; both are faster than this code. I wrote the pointers by hand because the assignment is about the mechanics, and because the invariant worth testing — that the dictionary and the list describe the same keys in the same order — is invisible behind `move_to_end`.

1.2 The cache implementation

The complete module follows — the same file, unedited, is `lru_cache.py` in the project. `__slots__` keeps the per-entry overhead down; the helpers at the bottom exist for tests and debugging and are not part of the hot path.

LRU_CACHE.PY

```
"""Problem 1 - Least Recently Used (LRU) cache with O(1) get and put.

Design: a dict maps every key to its node, and a doubly linked list keeps the
nodes in recency order (front = most recently used, back = least recently
used).  Each operation touches only a constant number of pointers.
"""

from __future__ import annotations

from typing import Dict, Iterator, Optional


class _Node:
    """One cache entry.  `__slots__` keeps the per-entry memory cost low."""

    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key: int = 0, value: int = 0) -> None:
        self.key = key
        self.value = value
        self.prev: Optional["_Node"] = None
        self.next: Optional["_Node"] = None


class LRUCache:
    """Fixed-capacity cache that evicts the least recently used entry.

    List layout (sentinels are never real entries):

        head <-> [most recent] <-> ... <-> [least recent] <-> tail
    """

    def __init__(self, capacity: int) -> None:
        if isinstance(capacity, bool) or not isinstance(capacity, int):
            raise TypeError("capacity must be an integer")
        if capacity <= 0:
            raise ValueError("capacity must be >= 1")

        self._capacity = capacity
        self._map: Dict[int, _Node] = {}

        # Two sentinels remove every null-check from the pointer surgery.
        self._head = _Node()
        self._tail = _Node()
        self._head.next = self._tail
```

```
        self._tail.prev = self._head

    # ------------------------------------------------------------------
    # Internal list primitives - each one is O(1)
    # ------------------------------------------------------------------
    def _detach(self, node: _Node) -> None:
        """Cut `node` out of the list, leaving its neighbours joined."""
        previous, following = node.prev, node.next
        assert previous is not None and following is not None, "sentinel detach"
        previous.next = following
        following.prev = previous
        node.prev = None
        node.next = None

    def _push_front(self, node: _Node) -> None:
        """Insert `node` directly after the head sentinel (most recent)."""
        first = self._head.next
        assert first is not None, "head sentinel is unlinked"
        node.prev = self._head
        node.next = first
        first.prev = node
        self._head.next = node

    # ------------------------------------------------------------------
    # Public API
    # ------------------------------------------------------------------
    def get(self, key: int) -> int:
        """Return the value for `key`, or -1.  Marks the key as used."""
        node = self._map.get(key)
        if node is None:
            return -1

        if node is not self._head.next:     # already most recent -> no work
            self._detach(node)
            self._push_front(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        """Insert or update `key`, evicting the LRU entry when full."""
        node = self._map.get(key)

        if node is not None:                # update in place, refresh recency
            node.value = value
            if node is not self._head.next:
                self._detach(node)
                self._push_front(node)
            return

        if len(self._map) == self._capacity:
            lru = self._tail.prev           # the oldest real node
            assert lru is not None, "tail sentinel is unlinked"
            self._detach(lru)
            del self._map[lru.key]

        node = _Node(key, value)
        self._map[key] = node
        self._push_front(node)

    # ------------------------------------------------------------------
    # Introspection helpers (used by the tests and the demo)
    # ------------------------------------------------------------------
    def __len__(self) -> int:
        return len(self._map)
```

```
    @property
    def capacity(self) -> int:
        return self._capacity

    def __contains__(self, key: object) -> bool:
        return key in self._map

    def keys_mru_to_lru(self) -> Iterator[int]:
        """Yield keys from most to least recently used - O(n), debug only."""
        current = self._head.next
        while current is not self._tail:
            assert current is not None
            yield current.key
            current = current.next

    def __repr__(self) -> str:
        entries = ", ".join(
            f"{k}: {self._map[k].value}" for k in self.keys_mru_to_lru()
        )
        return f"LRUCache(capacity={self._capacity}, [{entries}])"
```

1.3 The cases that break naive versions

Most incorrect LRU caches are wrong in one of these five places. They are listed here because each one is a test in the suite, not because the code is defensive.

SITUATION	CORRECT BEHAVIOUR	HOW THIS CODE GETS THERE
`put` an existing key while full	Replace the value, refresh recency, evict nothing	The update branch returns before the capacity check, so the size never changes.
`put` a new key while full	Evict exactly one entry, then insert	The check is `len == capacity` and happens before the insert; `tail.prev` is unlinked and its key removed from the map.
`get` a key that is already at the front	Return the value; order is unchanged	Relinking is skipped when the node is already `head.next`. Correct either way, but it saves four pointer writes on a hot path.
Capacity of one	Every new key evicts the previous one	Sentinels make the single real node both `head.next` and `tail.prev`; no special case is needed.
Zero or negative capacity	Reject at construction	Every `put` would evict the entry it just inserted, so a zero-capacity cache cannot honour its own contract. It raises `ValueError`.
Stored value of -1	Indistinguishable from a miss	That is the specified API. `key in cache` is available when a caller needs to tell the two apart.

1.4 Verification of the cache

47 tests, all passing. The ones that carry the most weight:

- the exact sequence from the brief, checked step by step;
- a sequence that a FIFO queue would also pass, followed by one that only LRU survives — this is the test that catches “evict the oldest inserted” instead of “evict the oldest used”;
- update-existing-while-full, and capacity one;
- 300 randomised caches × 200 random operations, compared after *every* operation against an independent `OrderedDict` implementation: return value, size and the full recency order.

The randomised comparison is the one I trust most. It reaches update-then-evict orderings that are tedious to enumerate by hand, and any divergence in the pointer surgery shows up immediately as a mismatch in the recency order.

Constant cost is also measurable. The table below is from `benchmark.py` on this machine (CPython 3.10, single thread):

WORKLOAD	RESULT
400,000 mixed get/put, capacity 10,000	0.390 s — about 1.03 million operations per second
200,000 operations at capacity 100	790 ns per operation
200,000 operations at capacity 10,000	922 ns per operation
200,000 operations at capacity 200,000	890 ns per operation
10,000 live entries	121 bytes per entry, including the dictionary slot and the node

Flat per-operation cost across a 2,000× range of capacities is the practical signature of O(1). The small drift is the dictionary's cache behaviour at larger table sizes, not a growing algorithm.

SECTION 2

Problem 2 — Event Scheduler

Both functions answer the same underlying question — how many events are alive at once — so both are built on one sorted sweep and one convention.

> **THE SHORT VERSION**
>
> Sort by start time and sweep once. The overlap check keeps the furthest end seen so far; the room count keeps a min-heap of end times, and its maximum size is the peak concurrency. The peak is the answer, because no schedule can beat it and a greedy one never exceeds it.

2.1 One convention: half-open intervals

Every event is treated as the half-open interval `[start, end)`. The convention does two things at once. It encodes the brief's rule that an event ending at 10 and an event starting at 10 do not conflict, and it turns "is this room free?" into one comparison, `end <= start`, with no special case anywhere. Both functions share the convention, so a schedule that `can_attend_all` accepts is a schedule `min_rooms_required` counts as one room. They cannot disagree.

2.2 `can_attend_all`: is the diary free?

Sort by start time, then make a single pass while remembering `furthest_end`, the latest moment at which the person is still busy. An event can be attended only if it starts at or after that moment. If it starts earlier, it collides with something already seen. After each event, `furthest_end = max(furthest_end, end)`, so the value never moves backwards even when a later event finishes sooner.

Keeping the running maximum, rather than comparing each event with its immediate predecessor, is what makes nesting free: in `[(9, 12), (10, 11)]` the second event ends before the first, but it starts before

`furthest_end = 12` and is caught without a separate containment rule. The strict comparison `start < furthest_end` is the adjacency rule: equal values are allowed.

Complexity. O(n log n) time, dominated by the sort; O(n) space for the sorted copy. An empty list returns `True` — nobody has a conflict.

2.3 `min_rooms_required`: count the peak

The key observation is that the answer is the peak concurrency. If *k* events are in progress at the same instant, no schedule can use fewer than *k* rooms, because each of them needs a distinct room at that instant. That is the lower bound, and it is easy. The upper bound is the useful half: a greedy sweep never opens a *k+1*-th room, because it only opens a room when every existing room is busy — and at that moment the new event plus the ones occupying those rooms are all in progress together, so the peak is at least the number of rooms it is about to exceed. If the peak is *k*, greedy never needs more than *k*. Minimum and maximum concurrency are the same number.

The implementation keeps a min-heap of end times for the rooms in use. Events are processed in start order. Before seating an event, every room whose end is at or before the current start is popped — because the heap is ordered by end time, one `while` loop releases all of them. Then the current end is pushed. The heap now holds exactly the events in progress at this start time, so its size is the concurrency at that instant.

EVENT	HEAP BEFORE	RELEASED	HEAP AFTER	ROOMS IN USE
`(0, 30)`	`[]`	—	`[30]`	1
`(5, 10)`	`[30]`	—	`[10, 30]`	2
`(15, 20)`	`[10, 30]`	`10`	`[20, 30]`	2

Peak is 2, and two rooms really are enough: the all-day event keeps one room while the two short events share the other. For the brief's `[(9,10), (10,11), (11,12)]`, every event releases its room before the next one starts, so the heap never holds more than one entry and the answer is 1.

2.4 The scheduler implementation

The module is `event_scheduler.py`. It validates that no event ends before it starts, and returns the schedule in chronological order so the output is stable regardless of input order.

EVENT_SCHEDULER.PY

```
"""Problem 2 - Event scheduling on half-open intervals.

Every event is treated as the half-open interval [start, end).  That single
convention encodes the requirement that an event ending at 10 and another
starting at 10 do not conflict, and it makes the sweep logic easy to reason
about.
"""

from __future__ import annotations

import heapq
from typing import Iterable, List, Sequence, Tuple

Interval = Tuple[float, float]


def _validated(events: Iterable[Interval]) -> List[Interval]:
```

```
    """Return the intervals as a list, rejecting reversed or empty input."""
    cleaned: List[Interval] = []
    for start, end in events:
        if start > end:
            raise ValueError(f"event end {end!r} precedes start {start!r}")
        cleaned.append((start, end))
    return cleaned


def can_attend_all(events: Sequence[Interval]) -> bool:
    """True when no two events overlap.

    Sort by start time and walk forward while remembering the furthest end
    seen so far.  A new event is safe only if it starts at or after that
    point.  Keeping the *maximum* end (instead of the previous end) is what
    makes nested intervals such as (9, 12), (10, 11) work.

    Time  O(n log n)  - the sort dominates
    Space O(n)        - the sorted copy
    """
    ordered = sorted(_validated(events))
    furthest_end = float("-inf")

    for start, end in ordered:
        if start < furthest_end:            # strict: touching is allowed
            return False
        furthest_end = max(furthest_end, end)
    return True


def min_rooms_required(events: Sequence[Interval]) -> int:
    """Minimum number of identical rooms needed to host every event.

    Sweep events in start-time order.  A min-heap holds the end time of every
    room currently in use; before seating the next event, every room that has
    already been released (end <= start) is popped.  Whatever remains is the
    set of concurrent events, and the largest such set over the whole sweep is
    exactly the number of rooms required.

    Time  O(n log n)  - one sort, n pushes, at most n pops
    Space O(n)        - the heap
    """
    ordered = sorted(_validated(events))
    busy: List[float] = []                  # min-heap of end times
    peak = 0

    for start, end in ordered:
        while busy and busy[0] <= start:    # room is free again
            heapq.heappop(busy)
        heapq.heappush(busy, end)
        peak = max(peak, len(busy))

    return peak


def room_label(index: int) -> str:
    """0 -> 'Room A', 25 -> 'Room Z', 26 -> 'Room AA', 27 -> 'Room AB'."""
    if index < 0:
        raise ValueError("room index must be >= 0")
    letters = ""
    n = index + 1                           # bijective base-26
    while n > 0:
        n, remainder = divmod(n - 1, 26)
        letters = chr(ord("A") + remainder) + letters
```

```
    return f"Room {letters}"


def assign_rooms(events: Sequence[Interval]) -> List[Tuple[float, float, str]]:
    """Greedily pin every event to a concrete room.

    Returns (start, end, room) tuples in chronological order. Reusing the
    lowest free room (or opening a new one) is optimal for interval
    partitioning, so the result uses exactly min_rooms_required(events) rooms.

    Time  O(n log n)
    Space O(n)
    """
    ordered = sorted(_validated(events))

    free: List[int] = []                            # min-heap of room ids
    busy: List[Tuple[float, int]] = []              # (end, room id)
    next_room_id = 0
    schedule: List[Tuple[float, float, str]] = []

    for start, end in ordered:
        while busy and busy[0][0] <= start:         # release finished rooms
            _, room_id = heapq.heappop(busy)
            heapq.heappush(free, room_id)

        if free:
            room_id = heapq.heappop(free)
        else:
            room_id = next_room_id
            next_room_id += 1

        heapq.heappush(busy, (end, room_id))
        schedule.append((start, end, room_label(room_id)))

    return schedule
```

A classical alternative avoids the heap entirely: sort starts and ends into two arrays and walk them with two pointers, opening a room when the next start precedes the current end and closing one otherwise. It is also O(n log n), with a smaller constant and the same O(n) space. I chose the heap because its contents are the live events, which is precisely the state needed to hand out concrete room numbers in 3.3; the two-pointer version counts rooms but forgets who is in them.

2.5 Verification of the scheduler

Beyond the examples in the brief, the suite covers adjacency, full nesting, duplicate events at the same time, unsorted input, empty input and reversed intervals. Two randomised checks carry the correctness argument:

- 400 random event sets compared against the $O(n^2)$ definition of overlap (two intervals conflict when `a.start < b.end` and `b.start < a.end`);
- 400 random event sets compared against a brute-force peak count, which samples the number of covering events at every start time.

Room assignment gets its own property test: over 200 random schedules, the number of distinct rooms must equal `min_rooms_required`, and no two events placed in the same room may overlap. Those two assertions together are the specification.

100,000 RANDOM EVENTS	TIME	RESULT
`can_attend_all`	0.087 s	`False`
`min_rooms_required`	0.167 s	15 rooms
`assign_rooms`	0.257 s	100,000 assignments

SECTION 3

Final Discussion & Analysis

Complexity first, then the trade-off behind the cache's two structures, then the two extensions: naming rooms, and sharing the cache between threads.

3.1 Time and space complexity

n is the number of events; *C* is the cache capacity. All bounds are for the implementation in this document.

Problem 1 — LRU cache

OPERATION	TIME	SPACE	WHY
`LRUCache(C)`	O(1)	O(1)	Two sentinels and an empty dictionary.
`get(key)`	O(1) expected	O(1)	One dictionary probe, then at most two pointer splices.
`put(key, value)`	O(1) expected	O(1)	One probe, one splice, and at most one eviction (also a splice).
Whole cache	—	O(C)	One dictionary entry and one node per live key; 121 bytes per entry measured.
`keys_mru_to_lru()`	O(C)	O(1) extra	Debug helper; it walks the list and is not used by the cache itself.
`ThreadSafeLRUCache.get/put`	O(1) + lock	O(1)	One mutex acquisition around the same work.
`ShardedLRUCache.get/put`	O(1) + shard lock	O(C)	Hash the key, lock one shard, do the same work.

"Expected" is the honest qualifier. The linked-list work is O(1) in the worst case, but the dictionary probe is O(1) only on average: adversarial keys can collide and degrade a probe to O(n). CPython randomises string hashes and uses open addressing, which makes that impractical for normal keys, but it is the reason a cache exposed to untrusted input should use a keyed or cryptographic hash rather than a predictable one. The two list primitives, `_detach` and `_push_front`, are O(1) each: a fixed number of pointer writes and no search.

Problem 2 — Event scheduler

FUNCTION	TIME	SPACE	WHY
`can_attend_all`	O(n log n)	O(n)	Sort dominates; one linear scan afterwards. The sorted copy is the O(n).
`min_rooms_required`	O(n log n)	O(n)	One sort, n pushes, at most n pops, each O(log n).
`assign_rooms`	O(n log n)	O(n)	Same sweep with two heaps; every room enters and leaves each heap at most once.
`room_label`	O(log n)	O(1)	Base-26 digits of the room index; at most a handful of divisions.

`_validated` is one O(n) pass and allocates the list it returns, so it does not change the bound. Sorting is the floor for both functions: comparison-based sorting is Ω(n log n), and any algorithm that needs global order inherits that bound. There is one escape worth knowing: if times are small integers — minutes in a week, for instance, where T = 10,080 — a counting or bucket sort replaces the comparison sort, giving O(n + T) time at the cost of O(T) memory. For a meeting platform that is effectively linear and often the right trade; for arbitrary timestamps, the O(n log n) version is the general one.

3.2 Trade-offs: why a hash map plus a doubly linked list

The pairing is not a trick; it is the only arrangement in which both operations are O(1) without extra bookkeeping. The two structures answer complementary questions, and each one is cheap only because the other exists. A dictionary alone can find a value but cannot find the oldest one without a scan. A list alone can find the oldest one instantly but cannot find a key without a scan.

STRUCTURE	LOOKUP	EVICT LRU	WHY IT IS NOT ENOUGH
Hash map only	O(1)	O(n)	No order; finding the oldest means scanning every entry, or storing timestamps and scanning for the minimum.
Array + hash map	O(1)	O(n)	Moving an entry to the front shifts every element after it.
Singly linked list + map	O(1)	O(n)	Unlinking needs the predecessor, and a singly linked node does not know it.
Hash map + doubly linked list	**O(1)**	**O(1)**	—
Hash map + min-heap by timestamp	O(1)	O(log n)	A log factor per eviction, a monotonic counter per entry, and no stable tie-breaking.
`OrderedDict`	O(1)	O(1)	The same design in C: faster, but it hides the mechanics and does not port to languages without an ordered map.

Why doubly linked and not singly linked. To remove a node from a singly linked list you need the node before it, and finding that node means walking from the head — O(n), which destroys the guarantee at exactly the moment it matters, during eviction. A `prev` pointer makes removal independent of position: four assignments, no search. Keeping a separate "predecessor" dictionary would also work, but it costs another hash lookup and more memory than a pointer that is already there.

What the combination costs. Each entry carries two pointers and a dictionary slot, so memory is roughly double a bare dictionary — measured at 121 bytes per entry here. Nodes are separately allocated, so a `get` touches a dictionary bucket, then the node, then its two neighbours: three or four cache lines in unrelated places. At ten thousand entries that is noise; at fifty million it is the dominant cost, and it is why high-throughput caches avoid a global linked list. The structure is also hostile to lock-free concurrency, because every eviction mutates shared neighbours (see 3.4).

Why not always exact LRU. LRU is a good policy for locality, not a universally good one. A sequential scan of one-off keys evicts everything useful, and against an adversarial access pattern LRU can be arbitrarily worse than Belady's offline optimum. Production systems therefore approximate it: CLOCK / second-chance keeps a reference bit instead of a list, 2Q and segmented LRU add a probationary segment so scans cannot flush the hot set, and Redis samples a handful of keys and evicts the oldest among them rather than maintaining a global order. Each of those gives up exactness for locality, memory or throughput — the same trade-off this exercise makes in the other direction.

3.3 Future proofing: assigning room numbers

Counting rooms is not the same as naming them. The sweep already knows which rooms are free at each start time, so the change is to carry identities instead of a count: one heap of free room ids, and one heap of `(end, room id)` for busy rooms. When an event starts, every room whose end is at or before that start moves from the busy heap to the free heap; the event then takes the smallest free id, or mints a new one. The result is deterministic, reuses the lowest-numbered room available, and uses exactly `min_rooms_required` rooms.

EVENT_SCHEDULER.PY — THE ASSIGNMENT LOOP

```
def assign_rooms(events: Sequence[Interval]) -> List[Tuple[float, float, str]]:
    """Greedily pin every event to a concrete room.

    Returns (start, end, room) tuples in chronological order. Reusing the
    lowest free room (or opening a new one) is optimal for interval
    partitioning, so the result uses exactly min_rooms_required(events) rooms.

    Time  O(n log n)
    Space O(n)
    """
    ordered = sorted(_validated(events))

    free: List[int] = []                          # min-heap of room ids
    busy: List[Tuple[float, int]] = []            # (end, room id)
    next_room_id = 0
    schedule: List[Tuple[float, float, str]] = []

    for start, end in ordered:
        while busy and busy[0][0] <= start:       # release finished rooms
            _, room_id = heapq.heappop(busy)
            heapq.heappush(free, room_id)

        if free:
            room_id = heapq.heappop(free)
        else:
            room_id = next_room_id
            next_room_id += 1

        heapq.heappush(busy, (end, room_id))
        schedule.append((start, end, room_label(room_id)))

    return schedule
```

The greedy is optimal for the same reason as the count: a new room is opened only when the free heap is empty, which means every existing room is busy at that instant, so the number of rooms in use is a lower bound on the peak. Choosing the smallest free id rather than an arbitrary one changes only the labels, never the number of rooms or the feasibility of the schedule.

Two details make it usable rather than merely correct. If the caller needs the answers in its own order instead of chronological order, sort by `(start, end, original index)` and write into a preallocated result list; that is a two-line change which does not touch the sweep. And labels are generated in bijective base-26, so the sequence continues past Z without collisions: `Room A … Room Z, Room AA, Room AB …`.

ACTUAL OUTPUT — PYTHON DEMO.PY

```
assign_rooms
  Room A   09:00-10:00   Kickoff
  Room B   09:00-11:00   Design review
  Room A   10:00-10:30   1:1 with Priya
  Room A   10:30-11:30   Customer call
  Room B   11:00-12:00   Retro
```

In a real platform the same sweep would need more state, and at some point greedy stops being enough:

- **Resources.** Projectors, capacity or accessibility turn room choice into a constraint-satisfaction problem, and the greedy assignment is no longer guaranteed optimal; a bipartite matching or a small ILP is the honest tool.
- **Buffers.** Ten minutes to reset a room is just an inflated end time, `end + buffer`, which the existing comparison already handles.
- **Time zones and DST.** Store UTC instants rather than wall-clock integers; a half-open interval over `datetime` behaves exactly like the integer version.
- **Recurring events.** Expand the recurrence rule over a bounded horizon, then schedule the expanded instances.
- **Fairness.** Replacing “smallest free id” with “least-used room” spreads wear and keeps premium rooms available; it is a different heap key, not a different algorithm.
- **Correctness under concurrency.** Two requests must not book the same room at once. A database exclusion constraint on `(room_id, tstzrange(start, end))` enforces the invariant where it matters, rather than trusting the scheduler process to be the only writer.

3.4 Concurrency: making the cache thread-safe

The first thing to notice is that `get` is not a read. A hit reorders the linked list, so every lookup is a mutation. That rules out the reflex answer of “wrap it in a readers-writer lock”: every `get` would need the write lock, and the lock would serialise exactly as much as a plain mutex while adding overhead.

Without a lock, several failure modes are reachable. Two threads can both observe a full cache and each evict a different node for one insertion, so the cache silently shrinks. A node can be detached twice, leaving its neighbours pointing at each other incorrectly. A thread can hold a reference to a node that another thread has just evicted and reinsert it, producing a node that is in the list twice or a key that points to a node no longer in the list at all. The GIL does not prevent any of this: it switches between bytecodes, and the window between the dictionary probe and the four pointer writes is many bytecodes wide.

THREAD_SAFE.PY — THE MINIMAL CORRECT CHANGE

```
class ThreadSafeLRUCache:
    """One mutex around the whole cache - correct, simple, serialising.

    A readers-writer lock would not help: every hit reorders the linked list,
    so `get` needs exclusive access just as much as `put` does.
    """

    def __init__(self, capacity: int) -> None:
        self._cache = LRUCache(capacity)
        self._lock = threading.Lock()

    def get(self, key: int) -> int:
        with self._lock:
            return self._cache.get(key)

    def put(self, key: int, value: int) -> None:
        with self._lock:
            self._cache.put(key, value)

    def __len__(self) -> int:
        with self._lock:
            return len(self._cache)

    def snapshot(self) -> List[int]:
        """Keys from most to least recently used, copied under the lock."""
        with self._lock:
            return list(self._cache.keys_mru_to_lru())
```

One mutex around both operations is correct, simple, and enough for most services. It also has to cover compound operations: "get, and on a miss compute and put" is not atomic just because each half is. Keep the critical section short, and never call user code — a loader, a network call, a callback — while holding the lock.

When a single lock becomes the bottleneck, the practical answer is sharding, not finer-grained locking of the same list: the list is one global order, so every eviction touches nodes that other operations also touch, and per-node locks would need hand-over-hand locking across four nodes. Sharding gives each shard its own list, lock and slice of the capacity, so different shards never contend. The price is per-shard eviction, which makes the global policy approximate: a hot key that hashes to a cold shard can be evicted before a cold key in a hot shard. `hash(key) % shards` spreads typical keysets evenly, so this is usually acceptable — but it is a behavioural change, not just a performance one.

At larger scale the same idea appears in different clothes. Redis keeps a 24-bit access clock per key and evicts the oldest of a small random sample, which avoids a global list entirely. Caffeine records reads in a striped, lock-free buffer and replays them into the eviction policy asynchronously, so recency is eventually consistent in exchange for scalability. Both accept a small amount of policy error to remove the contention that an exact global order creates.

I verified the wrapper rather than assuming it: eight threads × 4,000 random operations, with `sys.setswitchinterval(1e-6)` to force frequent preemption, then a white-box check that the map and the list contain the same keys in the same order, that the size is exactly the capacity, and that every `prev`/`next` link is consistent. A stress test cannot prove the absence of a race; it can only make one likely to appear, which is why the invariant check is the part that matters.

One boundary worth naming: this lock protects threads inside one process. An `asyncio.Lock` is the coroutine equivalent but does not protect threads, and a cache shared between processes needs a shared server — Redis

or Memcached — where writes are serialised by construction.

SECTION 4

Appendix

4.1 Project layout and how to run

The solution modules have no dependencies outside the standard library. `pytest` runs the tests; `Pygments` is used only by the script that builds this PDF.

REPOSITORY LAYOUT

```
Avaloka_Assignment/
├── lru_cache.py              Problem 1 — no dependencies
├── event_scheduler.py        Problem 2 — no dependencies
├── thread_safe.py            Section 3.4 - locked and sharded wrappers
├── demo.py                   the output shown in 3.3 and 4.2
├── benchmark.py              the numbers in 1.4 and 2.5
├── build_report.py           rebuilds this PDF from the sources
├── tests/
│   ├── test_lru_cache.py
│   ├── test_event_scheduler.py
│   └── test_thread_safety.py
└── Vihan_Chorada_Avaloka_Assignment.pdf
```

COMMANDS

```
python -m pytest tests -q      # 47 tests
python demo.py                 # readable walkthrough
python benchmark.py            # timings quoted above
python build_report.py         # regenerates this document
```

4.2 Captured output

The blocks below are the unedited output of `python demo.py` on the run that produced this document. They are the same code that the tests exercise.

PYTHON DEMO.PY — THE CACHE

```
LRU cache, capacity = 2
  operation       result   MRU -> LRU    note
  ------------------------------------------------------------
  put(1, 1)            -   [1]
  put(2, 2)            -   [2, 1]
  get(1)               1   [1, 2]
  put(3, 3)            -   [3, 1]        evicted 2
  get(2)              -1   [3, 1]
  put(4, 4)            -   [4, 3]        evicted 1
  get(1)              -1   [4, 3]
  get(3)               3   [3, 4]
  get(4)               4   [4, 3]
```

PYTHON DEMO.PY — THE SCHEDULER

```
can_attend_all
  [(9, 10), (10, 11), (11, 12)]     True
  [(9, 11), (10, 12)]               False
  [(9, 12), (10, 11)]               False
  [(7, 10), (2, 4)]                 True

min_rooms_required
  [(9, 10), (10, 11), (11, 12)]     1
  [(0, 30), (5, 10), (15, 20)]      2
  [(1, 5), (1, 5), (1, 5)]          3
  [(1, 10), (2, 3), (4, 5)]         2

assign_rooms
  Room A   09:00-10:00   Kickoff
  Room B   09:00-11:00   Design review
  Room A   10:00-10:30   1:1 with Priya
  Room A   10:30-11:30   Customer call
  Room B   11:00-12:00   Retro
```

4.3 Test suite and benchmark

PYTHON -M PYTEST TESTS -Q

```
.............................................                            [100%]
47 passed in 2.00s
```

PYTHON BENCHMARK.PY

```
LRU cache, capacity 10,000
  400,000 mixed operations      0.390 s   1.03 M ops/s
  10,000 live entries           121 bytes/entry
LRU cost per operation, 200,000 ops each
  capacity 100                     790 ns/op
  capacity 10,000                  922 ns/op
  capacity 200,000                 890 ns/op
Scheduler, 100,000 events
  can_attend_all                0.087 s   False
  min_rooms_required            0.167 s   15 rooms
  assign_rooms                  0.257 s   100,000 assignments
```

Every code block in this document is included from the files above at build time, so the printed source and the tested source cannot drift apart. The measurements are from CPython 3.10 on the machine that built the PDF; they are here to show the shape of the cost, not to serve as a benchmark of any particular hardware.